Future Lens: Anticipating Subsequent Tokens from a Single Hidden State

Koyena Pal
Northeastern University
pal.k@northeastern.edu

Jiuding Sun
Northeastern University
sun.jiu@northeastern.edu

Andrew Yuan
UMass Amherst
awyuan@umass.edu

Byron C. Wallace
Northeastern University
b.wallace@northeastern.edu

David Bau
Northeastern University
d.bau@northeastern.edu

arXiv:2311.04897v1 [cs.CL] 8 Nov 2023

Abstract

We conjecture that hidden state vectors corresponding to individual input tokens encode information sufficient to accurately predict several tokens ahead. More concretely, in this paper we ask: Given a hidden (internal) representation of a single token at position t in an input, can we reliably anticipate the tokens that will appear at positions $\geq t + 2$? To test this, we measure linear approximation and causal intervention methods in GPT-J-6B to evaluate the degree to which individual hidden states in the network contain signal rich enough to predict future hidden states and, ultimately, token outputs. We find that, at some layers, we can approximate a model's output with more than 48% accuracy with respect to its prediction of subsequent tokens through a single hidden state. Finally we present a "Future Lens" visualization that uses these methods to create a new view of transformer states.

1 Introduction

Do hidden states in large language models (LLMs) encode tokens farther than a single token ahead? If so, how can we decode this sequence of tokens from a single state? In this work we empirically investigate these questions using GPT-J-6B (Wang and Komatsuzaki, 2021). We train models to predict hidden states several tokens ahead of a given position t based *only* on a contextualized representation of the input at this position.

Auto-regressive transformer language models are typically trained to predict one token ahead, but recent work has hinted that individual hidden states may contain more information than just probabilities of the following token. For example, Meng *et al.* (2022a) trace information flow from subject tokens to associated attribute predictions many steps ahead. Elsewhere, Gurnee *et al.* (2023) suggest that neurons in early layers are dense with information, while middle layers have dedicated neurons that represent high-level contextual features.

Other related efforts have passed hidden intermediate states directly to the decoder head (skipping in-between layers) to "verbalize" such embeddings (Din et al., 2023; Belrose et al., 2023; nostalgebraist, 2020). Studies of memorization (Carlini et al., 2021, 2023, 2019) have identified the presence of very long memorized sequences generated by language models, and Zhang and He (2020) shows that progressively dropping layers during computation can still achieve a similar prediction output of the model when compared against their fully computed model run.

In this work we ask: To what extent can we extract information about future (beyond subsequent) tokens from a single hidden token representation? To answer this, we conduct three experiments. First, extending the ideas of Tuned Lens (Belrose et al., 2023; Din et al., 2023) and the Logit lens (nostalgebraist, 2020), we train linear models to approximate future model predictions several tokens in the future, in order to reveal the extent to which individual hidden states may directly encode subsequent tokens. Second, we perform a causal intervention study in which we transplant individual hidden states from one context to a completely different context and measure the extent to which future tokens that were predicted in the original context can be predicted in the foreign context. Finally, we fit a "soft prompt" to explicitly learn an optimal prompt that permits reading out information about subsequent tokens from a hidden state. Our code and data is available at https://future.baulab.info

2 Methods

To unveil the information about "future" tokens implicitly encoded in a single transformer state vector, we develop and compare several methods for predicting future tokens from a single hidden state. Each of our methods has the same goal: Extract accurate predictions of a model's probability distribution several tokens ahead, based on the in-

formation in only one hidden state at a single layer at one token of the transformer.

For our evaluations we use an autoregressive transformer (Vaswani et al., 2017) language model defined as a function $G : X \to Y$ over vocabulary V of size $|V| = d_v$. G takes in a sequence of tokens $x = [x_1, ..., x_T] \in X, x_i \in V$ and maps this to a probability distribution $y_T \in Y \subset [0,1]^{d_v}$, which (greedily) predicts the next-token $x_{T+1} = \text{argmax}\, y_T$. To generate additional tokens, the top predicted token x_{T+1} is added to the sequence of tokens $[x_1, ..., x_T, x_{T+1}]$ and the process is repeated until the next N tokens are produced.

To calculate each predicted probability distribution from an input sequence x, the transformer performs a sequence of computations at L layers; this can be decomposed as:

$$G(x) = D(b_L(\cdots(b_2(b_1(E(x))))\cdots)) \quad (1)$$

Where the first step $E :\to \mathbb{R}^{d_h}$ embeds each input token into an initial hidden representation, $e(x_i) = h_i^0 \in \mathbb{R}^{d_h}$; each layer $b_l : \mathbb{R}^{d_h \times T} \to \mathbb{R}^{d_h \times T}$ transforms the sequence of representations; and the decoder $D : \mathbb{R}^{d_h} \to Y$ decodes the predicted probability distribution $y_T = D(h_T^L)$ from the last layer at the last token. We write the output of layer l as $H_l = b_l(H^{l-1})$, where:

$$H^l = (h_1^l, ..., h_T^l) \in \mathbb{R}^{d_h \times T} \quad (2)$$

When generating a sequence of tokens beyond the given starting prefix of length T, we write:

$$y_{T+i} = G([x_1, .., x_{T+i-1}, x_{T+i}]) \quad (3)$$

$$x_{T+i+1} = \text{argmax}\, y_{T+i} \quad (4)$$

Our goal is to devise methods that can anticipate what G will predict for y_{T+1} through y_{T+N} from only a single hidden state at h_T^l.

2.1 Direct Vocabulary Prediction

Let h_T^l denote the hidden representation induced by G for token x_T at intermediate layer $l \leq L$, and let y_{T+N} denote the subsequent-token distribution predictions produced by G after token x_{T+N}. To predict y_{T+N} from h_T^l alone, we train a linear model g_θ to predict logits $\hat{z}_{T+N}$ that approximate $\hat{y}_{T+N}$ after softmax:

$$\hat{z}_{T+N} = g_\theta(h_T^l) \quad (5)$$

$$\hat{y}_{T+N} = \text{softmax}(\hat{z}_{T+N}) \approx \hat{y}_{T+N}$$

Since this model directly predicts the subsequent predictions over the full vocabulary from h_T^l, we call it the direct vocabulary prediction model.

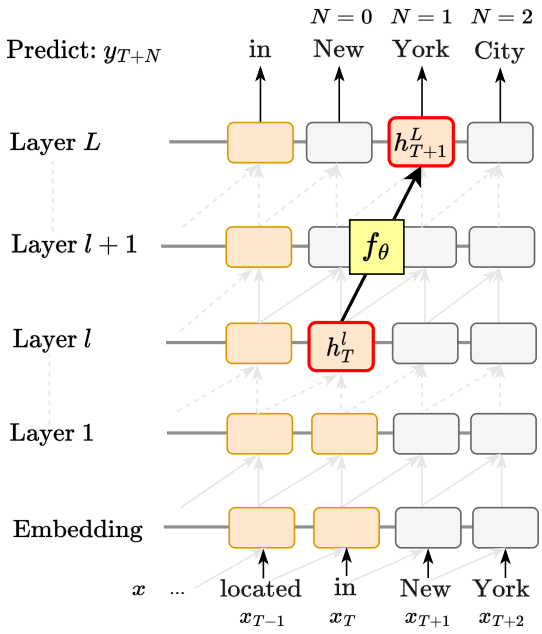

Figure 1: LLM to Linear Model Approximation Overview. Given a hidden state, h_T^l, the linear model, f_θ, is trained to output a future hidden state h_{T+1}^L. In this example, h_T^l is the encoding that would lead to the prediction of 'New,' and f_θ uses only that information to predict h_{T+1}^L that would predict 'York.'

2.2 Linear Model Approximation

We also test a linear model based on the tuned logit lens (Belrose et al., 2023; Din et al., 2023) approach, which anticipates future hidden states within the transformer and decodes them using the pretrained decoder head. Differently from that work, we model hidden states at future tokens in rather than only at later layers.

Beginning with the hidden representation h_T^l, we create a model to predict a hidden state h_{T+N}^L at the final layer L, and subsequent token x_{T+N}. To predict h_{T+N}^L from h_T^l, we train a linear model:

$$\hat{h}_{T+N}^L = f_\theta(h_T^l) \approx h_{T+N}^L \quad (6)$$

The vocabulary can be read from the predicted $\hat{h}_{T+N}^L$ by applying the pretrained decoder head of the transformer. In Figure 1, we show an example of one such linear model. Suppose that we have trained a linear model parameterized by θ, f_θ, that takes in the last token hidden representation of the input at layer l to generate a hidden state at layer L of the following token hidden representation. When we input the following in G: "Madison Square Garden is located in", we get "New" as the highest-probability prediction at $N = 0$ and "York"

at $N = 1$. We use the linear model to approximate this based on the hidden representation of T_N (i.e., "in") at layer $l \leq L$ as our input; the ideal output of the linear model given this would be the hidden state at T_{N+1} and layer L, which is associated with predicting "York" as the most probable token.

This approach differs from the direct vocabulary approach by reusing the pretrained decoder head of the transformer. We find that this marginally aids predictions at the latest layers l near L. Based on the observation that other pretrained transformer parameters may encode memorized calculations that facilitate decoding of subsequent tokens, we next turn to other approaches that utilize larger portions of the pretrained transformer to predict future tokens.

2.3 Fixed Prompt Causal Intervention

The next method we consider involves a single-state causal intervention where we transplant the hidden state h_T^l into the transformer while it is decoding an unrelated bit of context. The question is whether this transplantation steers the model to generate tokens related to the prefix that induced h_T^l. If it does, this indicates that information about subsequent tokens (in the original sequence) is prominently encoded in h_T^l.

Figure 2 depicts the procedure. On the left, we show the original context from which h_T^l is read; here $x = [x_1, ..., x_T]$ is "Madison Square Garden is located in" where x_1 is "Madison" and x_T is "in". This results in a sequence of outputs $[x_{T+1}, ..., x_{T+N}]$ which will read "New York City." On the right, we run a single generic fixed-context prompt $c = [c_1, ..., c_M]$ (e.g., "Please, tell me something about" where c_1 is "Please" and c_M is "about") through the transformer. One would not anticipate that this generic prompt would cause the transformer to predict "New York City".

Using an intervention, we now directly test that hypothesis that a single hidden state at layer l and token T within the original run contains the information necessary to predict subsequent tokens. We transplant the original run's state vector h_T^l into the corresponding location h_M^l in the fixed-context run, then allow the transformer to proceed. If the necessary contextual information is present in the new run, the resulting tokens generated would become "New" for the current token generation and "York" and "City" for the subsequent token generations.

Formally, let the sequence $x = [x_1, ..., x_T]$ denote an input context that causes the model to subsequently generate $[x_{T+1}, ..., x_{T+N}]$, and let and $c = [c_1, ..., c_M]$ represent a generic fixed-context prompt where T and M represent the lengths of the original and fixed input prompts, respectively. When each are passed through G, we get the following predicted distributions:

$$y_T = G(x) \in [0,1]^{|V|} \qquad (7)$$
$$\hat{y}_M^* = G(c) \in [0,1]^{|V|}$$

Denote the intervention that replaces h_M^l from the fixed-context run with state h_T^l from the original run as:

$$\hat{y}_M = G(c \,||\, h_M^l := h_T^l) \qquad (8)$$

If, after the intervention, the new predicted distribution $\hat{y}_M \approx y_M$ approximates the prediction in the original context, that will reveal that h_T^l specifically encodes information needed for that prediction.

Furthermore, we can deduce what h_T^l encodes about subsequent token predictions n steps ahead by adding the generated tokens to the input and comparing the following predictions:

$$y_{T+i} = G(x + [x_{T+1}, ..., x_{T+N}]) \qquad (9)$$
$$\hat{y}_{M+i} = G(c + [x_{T+1}, ..., x_{T+N}] \,||\, h_M^l := h_T^l)$$

The context prompt c could be chosen as any sequence of tokens. In practice, some prompts are more amenable to this intervention than others. In our experiments, we will test a small set of highly generic phrases.

2.4 Learned Prompt Causal Intervention

In the previous section, we have described an intervention that could reveal information predictive of upcoming tokens encoded in a single hidden state, by steering generation when grafted into completely unrelated contexts.

However, in cases where this "fails", it does not necessarily mean that the hidden state does not encode similar information; it may just be less prominent. To evaluate the degree to which such signal is present in these cases, we next explore an approach in which we *learn* to surface information about subsequent tokens from individual contextual token embeddings. This procedure is shown in Figure 3.

Specifically, we optimize a parameterized prefix, $c_{opt} = [c_1, ..., c_M]$ to extract this information from

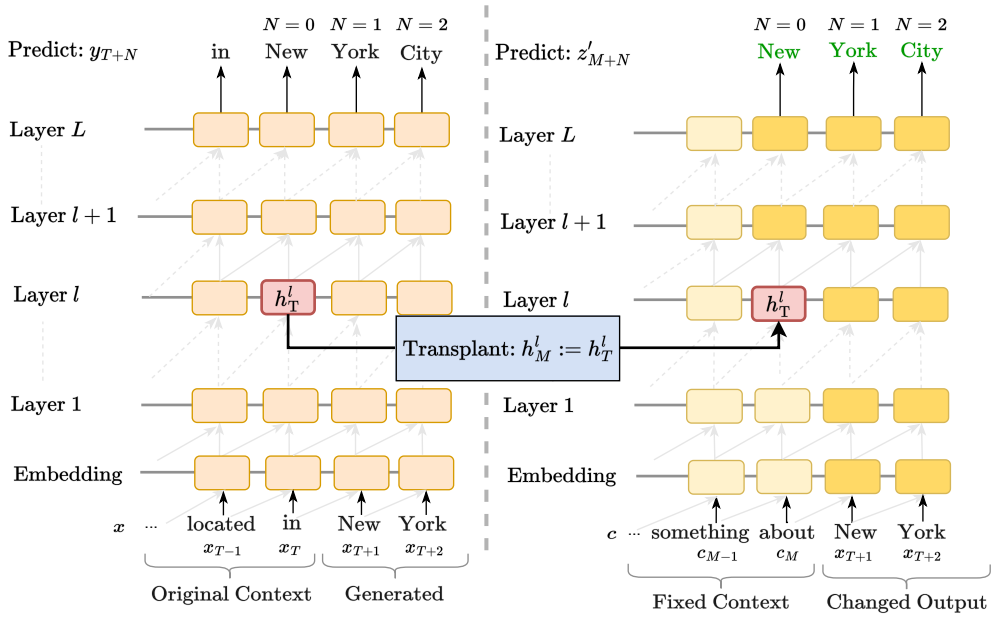

Figure 2: Illustration of Fixed prompt Causal Intervention. The left and right sides represent two different transformer model runs. On the left hand side, we have the original run of *Madison Square Garden ... in New York*. We transplant the hidden state, h_T^l to the other transformer model run, which has a fixed generic context, *Tell me something about*, as its input. With h_T^l replacing the hidden state at h_M^l, we measure the tendency of this modified transformer run to reveal the probability distribution in h_T^l. In such cases, it would reveal that h_T^l was predicting, for instance, 'New York City.'

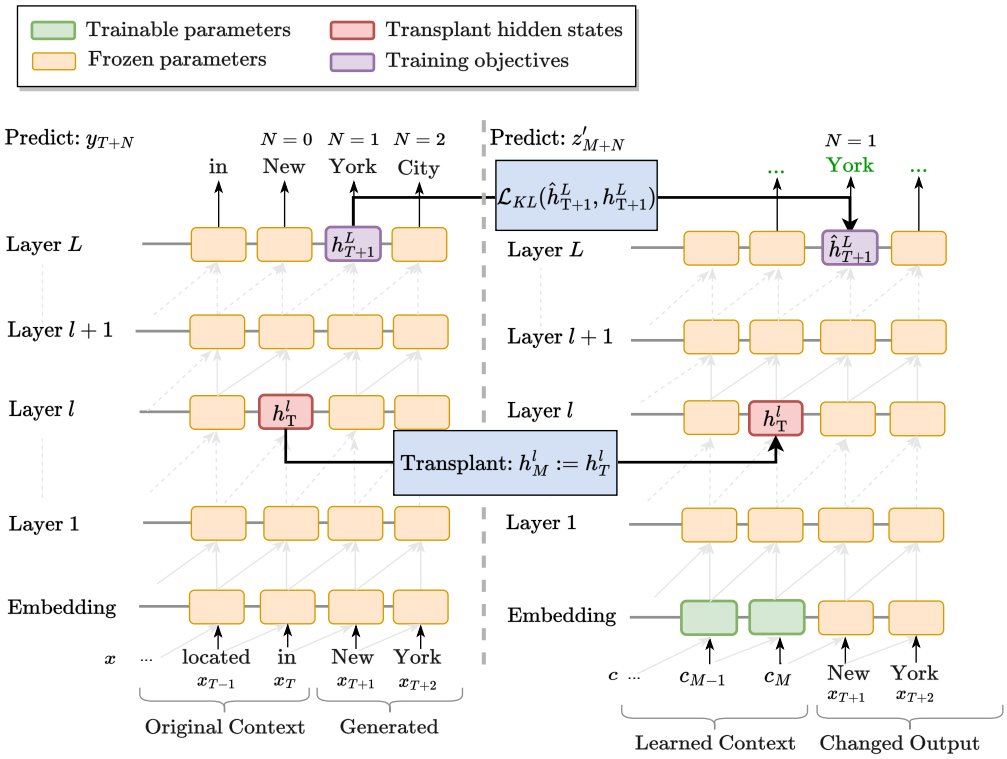

Figure 3: Learned context prompt Causal Intervention Overview. The left and right sides represent two different transformer model runs. The general setup is the same as Figure 2. The difference lies in the context provided in the transformer run on the right hand side. Instead of manually thinking of a context, we provide a learned context to increase the tendency of decoding the subsequent tokens predicted by h_T^l. We do so by training the context, c, with L_{KL} criterion and the objective to match the subsequent token prediction, such as 'York' in this instance.

the hidden state. For each decoder layer l, we train the corresponding prefix $c_{\text{opt}}^{(l)} = [c_1^{(l)}, ..., c_M^{(l)}]$ to maximize the probability of the model yielding the exact subsequent phrase after the original context. In particular, we conduct the same causal intervention in the hidden states h_T^l. We then optimize the probability distribution of the subsequent generation under the learned context to be the same as the original model when all its previous generation is given correctly:

$$\text{argmin}\, \text{KL}(\hat{y}_{M+N}\, ;\, y_{T+N}) \tag{10}$$

Where the predicted distribution $\hat{y}_n$ is given using the same intervention as described in Eq. 9:

$$\hat{y}_{M+n} = G([c_1, ..., c_M, x_{T+1}, .., x_{T+N}] \\ ||\, h_M^l := h_T^l) \tag{11}$$

We hence optimize this objective with the model frozen and only prefix left to be trained. Notably, our approach is different from the implementation of prefix tuning (Li and Liang, 2021) in the sense that we back-propagate the gradient through the model instead of a temporary MLP, as empirically it produces a significantly better optimized context.

3 Experiments and Results

3.1 Data

We perform evaluation on samples of the Pile (Gao et al., 2020), which is the 825GB dataset used to train GPT-J-6B (Wang and Komatsuzaki, 2021) as well as other LLMs.

To train the linear models, we sample 100,000 tokens that have an average of 518 sized-context. Amongst the 100,000 token samples, we use 10,000 of them to train for our learned prompt experiment. For testing our methods, we sample another 1000 tokens that have an average previous context length of 535. To simplify our analysis of the degree to which single hidden token representations encode subsequent n-grams, we draw our samples from contexts in which the original transformer model made a correct prediction.

More specifically, we randomly sampled train and test data points from the subset of token locations where the autoregressive transformer under consideration correctly predicts the following token. In Table 1, we break down the types of tokens present in the testing data by categorizing the last token (T) of the prefix as well as the generated tokens outputs of GPT-J , through greedy (argmax) decoding, at $N = 0, 1, 2, 3$ with respect to various properties, such as whether they are lower-cased tokens that start with a space, or are numerical tokens, and so on.

3.2 Evaluation Metrics

For evaluation we adopt the same metrics used in prior related work Din et al. (2023), namely Precision@k and Surprisal.

Precision@k measures the appearance of the top probability token in the output at N tokens ahead we predict from the hidden state with respect to the observed top-k tokens from GPT-J-6B model output. Higher values are better here because these mean the actual token at the corresponding future token was accurately predicted.

Surprisal, on the other hand, is the minus log probability according to the GPT-J-6B model output of the highest probability token according to the proposed probing methods. Lower is better for this measure because such values imply that the top predicted tokens are deemed probable by the model.

3.3 Experimental Setup

Linear Model We train two types of linear models — one with an output space of 4096 (the hidden representation size used by GPT-J-6B), and the other one with 50,400 (the vocabulary space of the same). GPT-J-6B comprises 28 layers. We train 4 instances for each of these layers, one for each different "future" token position we consider ($n = 0, 1, 2, 3$). As input we accept the source hidden state, i.e., h_T^l. Our output is either the hidden state, i.e., h_{T+N}^L or the decoded output at the position (vocabulary distribution) $T + N$.

Fixed Prompt Causal Intervention This is an evaluation-only setup where we choose four generic context prompts and perform causal intervention on these contexts as shown in Figure 2. The four fixed context prompts that we test are:

- `Hello! Could you please tell me more about "`
- `The multi-tokens present here are "`
- `The concepts in this hidden state listed are: (`
- `<|endoftext|> This state is describing about the following concept:`

The hidden states are gathered from layer l of the last token of the context tokens and are transplanted into the hidden representation of the last token in the generic prompts at the same layer l.